Module 3: Inferring Implicit Relationships

How a system connects data the question never explicitly links, and fills in what the user didn't say

Learning objectives

By the end of this module, you should be able to:

1. Model a data landscape as a **weighted schema graph**.
2. Score a candidate join using **value-overlap evidence**: containment, Jaccard and uniqueness.
3. Explain why edge costs use **−log P**, and find the most probable join path with a **shortest-path algorithm**.
4. Connect three or more entities at once with a **Steiner tree**.
5. Detect **fan-out and chasm traps**, where joins silently inflate numbers.
6. Measure **functional dependencies** with conditional entropy.
7. Infer **unstated constraints** (time windows, default exclusions) as latent variables.
8. Combine several weak signals with **chain products** and **noisy-OR**.

3.1 The problem: the question doesn't say how things connect

Consider: *"What was revenue by product category for customers in each region?"*

The user named four concepts (Revenue, ProductCategory, Customer, Region) but said **nothing** about:

- Which tables hold them
- How those tables link together
- What time period is meant
- Whether test accounts or cancelled orders count

Modules 1 and 2 found the right sources and concepts. This module covers the **hidden structure** between them.

We split "implicit" into two kinds:

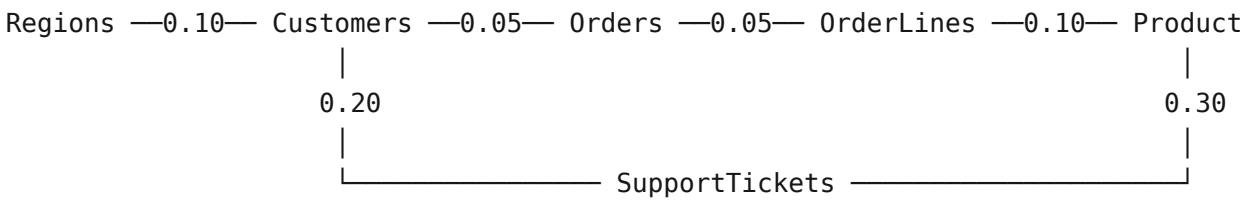
Kind	Question	Tools
Structural	How do these entities connect?	Schema graphs, join evidence, shortest paths, Steiner trees
Contextual	What constraints did the user assume?	Latent variables, priors, decision rules

3.2 The schema graph

Definition

```
H = (V, E, w)
  V = entities / tables          e.g. Customers, Orders, OrderLines, Products, Regions
  E = candidate links            pairs of columns that could join
  w : E → ℝ≥0                    cost of using a link (low = trustworthy)
```

Example landscape



Each number is an edge **cost**. The rest of this module explains where those costs come from and how to use them.

3.3 Evidence that two columns join

Column names are a weak hint. `cust_id` and `customer_id` *look* related, but so do `id` and `id` in two unrelated tables. Stronger evidence comes from the **values themselves**.

Let V(x) be the set of distinct values in column x.

Signal 1: Containment (inclusion dependency)

What fraction of x's values appear in y?

```
contain(x ⊆ y) = |V(x) ∩ V(y)| / |V(x)|
```

This measure is **asymmetric**. A foreign key x points to a primary key y when nearly all of x's values exist in y, even if y has extra values.

Signal 2: Jaccard similarity

```
J(x, y) = |V(x) ∩ V(y)| / |V(x) ∪ V(y)|
```

This measure is symmetric. It is lower when one side has many extra values, so containment is usually more informative for joins.

Signal 3: Uniqueness (key-ness)

```
uniq(y) = distinct values in y / rows in y          (1.0 = every value is unique)
```

When one side is unique, the join is **many-to-one**: safe for looking up attributes.

Signal 4: Name similarity

Edit distance or embedding similarity of column names (Module 2), after normalization.

Combining the signals: logistic model

```
P(valid join | x, y) = σ(z) = 1 / (1 + e^(−z))

z = θ₀ + θ₁·name_sim + θ₂·contain(x ⊆ y) + θ₃·uniq(y)
```

Illustrative weights: $\theta_0 = -4$, $\theta_1 = 3$, $\theta_2 = 4$, $\theta_3 = 2$.

Worked example: real join vs. coincidence

Candidate A: `Orders.cust_id` → `Customers.customer_id`

- Orders has 950 distinct cust_id values, and Customers has 1,000 distinct customer_id values. They share 940.
- contain = 940 / 950 ≈ **0.989**
- Jaccard = 940 / (950 + 1,000 − 940) = 940 / 1,010 ≈ **0.931**
- uniq(Customers.customer_id) = **1.0**
- name_sim ≈ **0.8**

```
z = −4 + 3(0.8) + 4(0.989) + 2(1.0) = −4 + 2.4 + 3.956 + 2 = 4.356
P = 1 / (1 + e^(−4.356)) = 1 / (1 + 0.0128) ≈ 0.987
```

Candidate B (decoy): `Orders.cust_id` → `Employees.emp_id`

Both are integer IDs, so some values overlap by coincidence.

- contain ≈ **0.30**, uniq = **1.0**, name_sim ≈ **0.3**

```
z = −4 + 3(0.3) + 4(0.30) + 2(1.0) = −4 + 0.9 + 1.2 + 2 = 0.1
P = 1 / (1 + e^(−0.1)) = 1 / (1 + 0.905) ≈ 0.525
```

> **Lesson:** both keys are unique integers, but only one shows **near-total containment**. Coincidental overlap between ID ranges is the most common source of false joins, which is why containment carries the largest weight.

3.4 Why edge cost = −log P

We want the **most probable** chain of joins. If the joins are independent, a path's probability is the **product** of its edge probabilities:

$$P(\text{path valid}) = \prod_{e \in \text{path}} P(e)$$

Taking -log turns products into sums, and maximization into minimization:

$$\operatorname{argmax}_{\text{path}} \prod P(e) \quad = \quad \operatorname{argmin}_{\text{path}} \sum -\log P(e)$$

So defining **cost(e) = −ln P(e)** means that **shortest path = most probable path**, and standard graph algorithms apply directly.

P(e)	cost = −ln P(e)
0.99	0.010
0.95	0.051
0.90	0.105
0.75	0.288
0.525	0.644
0.10	2.303

From 3.3: Candidate A costs ≈ 0.013, and Candidate B costs ≈ 0.644. The decoy is about **50× more expensive** to traverse.

> **Why not cost = 1 − P?** Sums of (1 - P) don't correspond to any probability of the whole path. Only -log composes correctly.

3.5 Finding the best join path

Dijkstra's algorithm (intuition)

Grow outward from the start node, always expanding the cheapest frontier node first. The first time the target is reached, the cost found is the cheapest possible.

```
dist[start] = 0;  dist[others] = ∞
repeat:
    u ← unvisited node with smallest dist
    for each neighbour v of u:
        dist[v] ← min( dist[v], dist[u] + w(u, v) )
until target visited
```

Worked example: Customers → Products

Using the graph in 3.2:

Path	Cost	P(path) = e^(-cost)
Customers → Orders → OrderLines → Products	0.05 + 0.05 + 0.10 = **0.20**	e^(-0.20) ≈ **0.82**
Customers → SupportTickets → Products	0.20 + 0.30 = **0.50**	e^(-0.50) ≈ **0.61**

The purchase path wins.

Paths carry meaning

Both paths are *valid* joins, but they answer **different questions**:

- Via Orders: *"products customers **bought**"*
- Via SupportTickets: *"products customers **complained about**"*

So path choice is also a semantic decision. Combine graph cost with fit to the question's intent:

```
score(path) = −cost(path) + λ · sim( concepts on path, C(Q) )
```

"Revenue" lives on OrderLines, so the purchase path gains a further bonus.

When to ask

```
ratio = P(best path) / P(second-best path)
if ratio < ρ  (e.g. 1.5)  →  paths are comparably plausible: clarify or state the choice
```

Here 0.82 / 0.61 ≈ 1.34. On graph cost alone that falls under ρ, but the semantic bonus from "revenue" resolves it.

3.6 Connecting many entities: Steiner trees

Our question needs **four** entities connected at once: {Regions, Customers, OrderLines, Products}. Joining pairs one at a time can produce redundant or conflicting paths. Instead, find the cheapest **tree** that touches every required node:

```
T* = argmin_{T ⊆ H, T connects all terminals}  Σ_{e ∈ T} w(e)
```

This is the **Steiner tree problem**. The tree may include extra "connector" nodes, such as Orders here, which the user never mentioned.

Solution for the example

```
Regions —0.10— Customers —0.05— Orders —0.05— OrderLines —0.10— Products

Total cost = 0.30     ⇒  P(all joins valid) ≈ e^(−0.30) ≈ 0.74
```

Orders is a **Steiner node**: it is required for connectivity, even though the question never names it.

Complexity

- The exact Steiner tree problem is **NP-hard**.
- **Approximation:** compute shortest paths between every pair of terminals, build a minimum spanning tree over those distances, then expand it back into real edges. This is guaranteed to be within **2×** the optimum, and usually much closer.

3.7 Join traps: when correct joins give wrong numbers

A join can be *valid* and still **inflate** results. This is one of the most common silent errors in business analytics.

Fan-out

Let order o have amount a_o and match m_o rows in a joined table.

$$\text{SUM over joined rows} = \sum_o a_o \cdot m_o$$
$$\text{correct total} = \sum_o a_o$$

This is correct **only if every $m_o = 1$**.

Example: order #1 is worth $100 and has 3 shipment rows. After the join, the sum reports **$300**.

Chasm trap

Here one parent has two independent one-to-many children.

A customer has 3 orders and 2 support tickets. Joining both onto the customer produces 3 × 2 = **6 rows**:

```
COUNT(orders) after join = 6   (true value: 3)
COUNT(tickets) after join = 6  (true value: 2)
```

In general, a parent with k_1 and k_2 children produces $\mathbf{k_1 \cdot k_2}$ rows.

Detection and fixes

```
fan-out factor  F = rows after join / rows before join
F > 1 on the table that holds the measure  ⇒  inflation risk
```

Fix	Idea
Aggregate first, then join	Sum each child to the parent grain before joining
Count distinct	COUNT(DISTINCT order_id) instead of COUNT(*)

Respect grain	Only join *up* (many-to-one) from the measure's table

> **Rule of thumb:** a measure should only travel along **many-to-one** edges. The uniqueness signal from 3.3 tells you which direction that is.

3.8 Hidden hierarchies: functional dependencies

Some relationships live **inside** a table, such as City → Region → Country. If each city belongs to exactly one region, then *region is a function of city*, and the system can roll up from city to region safely.

Measuring it with conditional entropy

$$H(Y \mid X) = \Sigma_x \, P(x) \cdot H(Y \mid X = x)$$

$$H(Y \mid X) = 0 \iff X \to Y \text{ is a perfect functional dependency}$$

A normalized version, the **uncertainty coefficient**:

$$U(Y \mid X) = 1 - H(Y \mid X) \,/\, H(Y) \qquad (1 = \text{perfect dependency}, 0 = \text{independent})$$

A simpler count-based alternative: **g_3 = the fraction of rows that must be removed to make the dependency exact.**

Worked example

City	Region
Paris	EU
Paris	EU
Lyon	EU
Austin	NA
Austin	NA
Austin	EU ← data error

$$H(\text{Region} \mid \text{Paris}) = 0 \qquad \text{(always EU)}$$
$$H(\text{Region} \mid \text{Lyon}) = 0$$
$$H(\text{Region} \mid \text{Austin}) = -\left(\tfrac{2}{3}\log_2\tfrac{2}{3} + \tfrac{1}{3}\log_2\tfrac{1}{3}\right) \approx 0.918 \text{ bits}$$

$$H(\text{Region} \mid \text{City}) = (2/6)(0) + (1/6)(0) + (3/6)(0.918) \approx 0.459 \text{ bits}$$
$$H(\text{Region}) = -(4/6\log_2 4/6 + 2/6\log_2 2/6) \approx 0.918 \text{ bits}$$

$$U = 1 - 0.459 / 0.918 = 0.50$$
$$g_3 = 1/6 \approx 0.17 \qquad \text{(remove one row → perfect dependency)}$$

Interpretation: the two metrics disagree in tone. The entropy measure looks weak at **0.50**, because tiny samples magnify a single error. g_3 shows the dependency holds for **83%** of rows, and the violation is one dirty record. In practice, a dependency with a small g_3 suggests "use the hierarchy, and flag the data-quality issue."

3.9 Contextual inference: what the user didn't say

Unstated constraints are **latent variables** z. We infer them the same way we inferred concepts:

$$z^* = \text{argmax}_z \; P(z \mid Q, \mathcal{O}) \quad \propto \quad P(Q \mid z) \cdot P(z \mid \text{intent, ontology})$$

Example 1: a standard exclusion

The ontology records a curated rule that business metrics exclude test accounts:

$$P(\text{exclude_test} = \text{true} \mid \text{metric} = \text{Revenue, curated rule}) \approx 0.95$$

Apply the rule by default and mention it.

Example 2: an unstated time window

"How has revenue been trending?" The intent is a **trend** (more in Module 4).

Candidate z	P(z | trend intent)
All time	0.20
Year to date	0.30
Last 12 months	**0.50**

The decision rule from Module 2 still applies:

$$P(z_1) = 0.50 < \tau = 0.6 \;\Rightarrow\; \text{not confident enough to apply silently}$$

Action: use the last 12 months, and **state the assumption explicitly** so the user can correct it.

Decision theory view

Choose the constraint that minimizes **expected loss**:

$$z^* = \text{argmin}_z \; \Sigma_{z'} P(z' \mid Q) \cdot \text{Loss}(z, z')$$

- With 0/1 loss, this reduces to "pick the most probable z".
- With **asymmetric loss**, for example when silently including test data is far worse than excluding it, the choice can move away from the most probable option toward the *safest* one.

Explicit constraints always win

$$\text{if the user states constraint } \kappa \text{ explicitly:} \quad P(\kappa) = 1, \quad \text{override every prior}$$

Priors only fill **gaps**. They never overwrite what the user actually said.

3.10 Combining weak evidence

Relationships are often supported by several imperfect clues.

Chains (all links must hold): multiply

$$(A\ R_1\ B) \wedge (B\ R_2\ C) \Rightarrow A \sim C \quad \text{with} \quad \text{conf} = \text{conf}(R_1) \cdot \text{conf}(R_2)$$

For example, 0.9 × 0.8 = **0.72**. Confidence shrinks with every hop, which is why long inferred chains deserve scepticism.

Independent clues (any one suffices): noisy-OR

$$P(\text{related}) = 1 - \prod_i (1 - p_i)$$

Example: a name match (0.6) and a value overlap (0.5) independently suggest the same link.

$$P = 1 - (1 - 0.6)(1 - 0.5) = 1 - 0.4 \cdot 0.5 = 0.80$$

Independent clues **reinforce** each other, while chained clues **dilute**.

Structure	Combination	Effect of adding evidence
Series (AND)	Product	Confidence ↓
Parallel (OR)	Noisy-OR	Confidence ↑

3.11 Putting it together: scoring a query plan

A complete **plan** π fixes the sources, joins, roll-ups and filters. Its overall score combines everything in this module:

$$\begin{aligned}\log \text{Score}(\pi) = {} & \sum_{e \in \text{Steiner tree}} \log P(e) && \leftarrow \text{join validity} && (3.3\text{–}3.6)\\ & + \lambda_1 \cdot \text{sim}(\text{path concepts}, C(Q)) && \leftarrow \text{path meaning} && (3.5)\\ & - \lambda_2 \cdot \mathbb{1}[\text{fan-out on the measure}] && \leftarrow \text{trap penalty} && (3.7)\\ & + \sum_{\text{FDs used}} \log(1 - g_3) && \leftarrow \text{hierarchy soundness} && (3.8)\\ & + \sum_{\text{latent } z} \log P(z^* \mid Q, \mathcal{O}) && \leftarrow \text{assumed constraints} && (3.9)\end{aligned}$$

The best plan is $\pi^* = \text{argmax} \log \text{Score}(\pi)$. Every assumption made along the way is **surfaced** to the user.

3.12 Common pitfalls

Pitfall	Symptom	Fix in the math
Trusting name matches	Joining `id` to `id` across unrelated tables	Weight containment and uniqueness above names
Coincidental ID overlap	Plausible but meaningless results	Require high containment, not just some overlap
Ignoring path meaning	"Bought" vs. "complained about" confused	Add semantic fit to the path score
Fan-out / chasm traps	Totals double or triple silently	Measures only travel many-to-one; aggregate first

Long inference chains	Confident but wrong connections	Chain products reveal decaying confidence
Silent assumptions	User unaware a filter was applied	State any latent z whose $P < \tau$
Priors overriding the user	User's filter ignored	Explicit constraints get P = 1

Check your understanding

1. **Compute:** a path has edges with P = 0.9 and P = 0.8. What are its total cost and its probability?
2. **Compute:** column A has 200 distinct values, B has 500, and they share 180. What are contain($A \subseteq B$) and Jaccard? Which column is more likely the foreign key?
3. **Compute:** an order worth $50 joins to 4 line-item rows. What does a naive SUM report?
4. **Compute:** two independent clues suggest a link with P = 0.7 and P = 0.4. What is the combined noisy-OR probability?
5. **Reason:** why must edge cost be -log P rather than 1 - P for shortest-path search to find the most probable path?
6. **Reason:** a user asks "revenue last quarter, including test accounts". What happens to the curated exclusion rule?

<details>
<summary>Answers</summary>

1. Cost = $-\ln 0.9 - \ln 0.8 \approx 0.105 + 0.223 =$ **0.328**. Probability = 0.9 × 0.8 = **0.72** (equivalently, e^(-0.328) ≈ 0.72).
2. contain = 180 / 200 = **0.90**. Jaccard = 180 / (200 + 500 - 180) = 180 / 520 ≈ **0.346**. **A** is likely the foreign key pointing to B: most of A's values live in B, while B has many extra values.
3. $50 × 4 = **$200**, inflated by a fan-out factor of 4.
4. 1 - (0.3)(0.6) = 1 - 0.18 = **0.82**.
5. Path probability is a **product** of edge probabilities. Only -log converts that product into a sum, so minimizing summed cost is the same as maximizing probability. Sums of (1 - P) have no such correspondence.
6. The explicit constraint gets **P = 1** and **overrides** the curated default. Priors fill gaps; they never contradict what the user stated.

</details>

Key takeaways

- Data landscapes are **graphs**. Joins are edges with cost **-log P(valid)**, so the **shortest path is the most probable path**.
- **Containment plus uniqueness** beat column names as join evidence.
- **Steiner trees** connect many entities at once, often through nodes the user never mentioned.
- Valid joins can still give wrong numbers: watch for **fan-out and chasm traps**, and let measures travel only many-to-one.
- **Conditional entropy and g_3** reveal hidden hierarchies and their data-quality issues.
- Unstated constraints are **latent variables**: fill gaps with priors, surface uncertain assumptions, and never override explicit instructions.
- Evidence in **series dilutes** confidence, while evidence in **parallel reinforces** it.

Coming next: Module 4, Intent Detection and Entity Resolution

Is the user asking for a trend, a ranking, a comparison or an explanation, and which "Paris" or "Apple" do they mean? We'll cover intent classification, mapping intents to query templates, choosing aggregation operators from modifier words, mention detection, candidate scoring, joint entity assignment, and abstaining when no candidate is good enough.

Report generated by Databricks Genie